

The Story of the Mitutoyo 52-Bit Data Stream

1. The Problem Emerges

In the 1990s, machinists and electronics tinkerers started wondering how to get measurements out of their fancy new Mitutoyo digital calipers and micrometers. These tools had a little port on the side — the SPC Digimatic interface — but no clear public instructions on how to talk to it.

People saw three mysterious wires: DATA, CLOCK, and REQ (request line). If you pulled REQ low, the caliper would twitch and then spit out a series of pulses on DATA, synchronized to CLOCK. The challenge: what do those pulses mean?

2. The First Maps (Don Lancaster, tinaja.com, 1999–2000)

Don Lancaster, the 'guru of hardware hacking,' wrote about this in his Tech Musings newsletters. In MUSE #142 (Nov 1999) and MUSE #145 (Feb 2000), he described using a PIC microcontroller to capture Digimatic data and convert it to RS-232 serial. He referenced Mitutoyo's official manual (No. 4330, 1997), which laid out the actual spec:

Mitutoyo Digimatic Data Format (Manual 4330)

- Each transfer = 52 bits = 13 nibbles (4 bits each)
- Transmission is LSB first
- Nibbles 0–7: 8 BCD digits (least significant digit first)
- Nibble 8: sign (bit 3 = 1 means negative)
- Nibble 9: decimal position (0–7)
- Nibble 10: units (0 = mm, 1 = inch)
- Nibbles 11–12: status (battery low, overrange, etc.)

In theory, that was the answer. In practice, Lancaster's implementation was buggy — it made assumptions about timing and bit order that didn't hold true across all devices.

3. Early Frustrations (2000s)

Hobbyists who tried to build from Don's article often failed. The PIC assembly code was finicky, and many calipers didn't quite match the assumptions. Timing was off, the sign sometimes came back wrong, and digits got scrambled.

As one Arduino forum member later put it:
'That article is fatally flawed.'

4. A Fresh Approach (Steve Spence, 2010–2011)

Around 2010–2011, Steve Spence ('sspence') developed a new Arduino-based solution. He published his work on Instructables: 'Interfacing a Digital Micrometer to a Arduino & VGA Monitor.'

His code directly implemented the 52-bit, 13-nibble read in a clear, C-based Arduino sketch. It added configurable REQ timing, reliable interrupt-driven bit capture, printing all 13 raw nibbles for verification,

and flexible decode of sign, decimal, and units. This implementation worked across different Mitutoyo and Igaging models.

5. Community Amplification (2011–2015)

Spence's work quickly spread. On the Arduino Forum (2011), he (as 'sspence65') helped others refine the protocol. Blogs like LearnArduinoNow published tutorials crediting his work, and machinist forums pointed newcomers to his project. Others extended it to Bluetooth, Raspberry Pi, and DRO systems — all using the 52-bit, 13-nibble mapping he clarified.

6. The Canonical Solution (2020s → Today)

By now, the sspence mapping has become community canon:

- 52 bits, 13 nibbles, LSB-first
- Digits in nibbles 0–7, sign in 8, decimal in 9, status in 11–12
- Use Arduino interrupts to capture on CLK edges

That is the foundation for all modern implementations.

7. Credits & Acknowledgements

Credits:

- Don Lancaster (tinaja.com, 1999–2000): first public notes on Digimatic, though flawed.
- Mitutoyo (Manual 4330, 1997): original 52-bit, 13-nibble specification.
- Steve Spence ('sspence', 2010–2011): robust Arduino implementation that became canonical.
- Arduino Forum & LearnArduinoNow (2011+): amplified and spread the work.
- Later hobbyists (2015+): built on this for wireless, Pi-based, and DRO systems.

Canonical Arduino Code

```
// Interrupt-driven Mitutoyo 52-bit reader (Arduino UNO/Nano)
// Based on the community-standard mapping: n0..n7 = digits, n8 = sign, n9 = decimal, n10 = units
// (sspence's original approach, refined and republished many times)

#define CLK_PIN    2    // INT0
#define DATA_PIN  4
#define REQ_PIN    3

// (Code omitted for brevity - see earlier full version)
```